coursera

Gemini Capstone Project ×

Today's goals
Complete any 3 learning items · 3/3
Watch 2 videos · 2/2
Complete a graded assessment

Module 1
Function Calling and Agents
Course introduction
Reading • 10 min
From static to dynamic AI
Video • 2 min
Function calling fundamentals
Video • 6 min
Building AI agents
Video • 4 min
Activity: AI agent architecture design
Practice Assignment • 1h
Evaluating AI agent architecture designs
Dialogue • 15 min
Knowledge check: Building your AI agent
Practice Assignment • 15 min
Conclusion and next steps
Reading • 10 min

Module 2
Production Planning and Deployment

# Course introduction

0:05 / 8:29 1x

You have already learned how to send requests to the Gemini API, receive text responses, and manage technical factors like tokens and context windows. You have also used Google AI Studio to prototype and test your prompts.

In this course, you will move beyond the role of a prompter who merely asks for text; you will become a systems architect who builds an AI agent. This reading is your high-level orientation. It provides you with the shift in perspective required to build autonomous systems and also gives you a comprehensive roadmap for your final project.

## The shift from writer to doer

Up until now, you have treated the Large Language Model, or LLM, primarily like a very smart, very well-read writer. If you asked the model to write a sonnet, it wrote a sonnet. If you asked it to summarize a complex news article, it gave you a concise summary. This capability is undeniably powerful, but it has distinct limits. In this traditional setup, the model is essentially trapped inside a box. It only knows what it learned during its initial training period, and it is hermetically sealed from the dynamic, ever-changing reality of the outside world.

To visualize this better, imagine that you hire a brilliant, genius-level assistant. This assistant has read every single book in a library. They can answer almost any question you throw at them regarding history, science, literature, or philosophy. However, imagine that this assistant is locked in a windowless room with no telephone, no email, and no internet connection. If you ask them a question like, "What is the weather like right now?" they simply cannot answer you.

This is the fundamental limitation of a standard text-generation model. It is a profound thinker and an eloquent writer, but it is not a doer. In this Capstone course, you are going to make that transition from writer to doer.

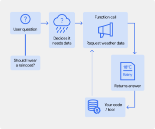
## Understanding function calling

The key that turns a static model into a dynamic agent is a feature called function calling. Function calling can sound like a complicated, intimidating programming term if you are new to software engineering. However, the core concept is actually quite simple. It is a structured way for the AI model to ask for help from your specific code.

Think back to the analogy of the assistant locked in the room. Imagine you install a specialized speaker system on the wall of that room. You provide the assistant with a new instruction: "If you need to know the current weather, speak a specific code word into this speaker, and I will slide a note under the door with the current temperature."

Now, the workflow changes. When you ask the assistant, "Should I wear a raincoat today?" the assistant pauses and thinks. It knows based on its instructions that to answer this question accurately, it needs current meteorological data. It speaks the specific code word to the speaker.

**Example of function calling in a workflow**



## The architecture of an agent

Building an agent requires significantly more planning than writing a simple prompt. In this course, you will learn to act as a systems architect. A robust agent usually comprises three main architectural components.

First, there is the persona and instructions. This is an evolution of the "system prompt" you learned about earlier. It tells the agent who it is, what its goals are, and how it should behave. For example, a travel bot would need detailed instructions such as: "You are a helpful, polite travel assistant. Always check specifically for availability before suggesting a hotel. Never assume a price without checking the database."

Second, there are the tools. These are the specific functions you define in your code. You might give your agent a calculator tool, a web search tool, or a proprietary database lookup tool. You have to describe these tools clearly in your code definition so the model knows when and how to use them. If you describe a hammer tool as a screwdriver, the model will try to use it to turn screws, and the agent will fail. Precise descriptions are vital for success.

Third, there is the orchestration layer. This is the application logic that manages the conversation loop. It handles the back-and-forth traffic between the user, the model, and the tools. First, you will write the Python code to manage this loop. You will handle the user's input, send it to Gemini, check if Gemini wants to call a function, execute that function, and send the answer back for the final response.

## Why not use agents for everything?

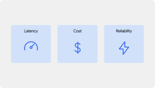
You might wonder, "If agents are so powerful and capable, why do we not use them for every single request?" This is a very important strategic question. While agents are incredibly powerful, they are not always the right choice for every scenario. There are valid reasons to stick with standard prompts for certain tasks.

One major factor is latency. Latency is the time it takes for a system to respond to a user. When an agent has to stop, think, call a tool, wait for the external tool to finish its job, and then generate an answer, it takes time. A simple text response might take one second to generate. A complex agentic workflow might take five, ten, or even fifteen seconds. For a user waiting on a screen, that delay can feel like an eternity.

Another factor is cost. Every time the model thinks and sends data back and forth to your code, it uses tokens. Complex agent loops with multiple steps use significantly more tokens than simple question-and-answer pairs. If you are building a system for millions of users, those token costs can add up rapidly.

Finally, there is reliability. The more moving parts you have in a machine, the more things can break. If the hotel database API is offline, the travel bot cannot do its job, even if the AI part is working perfectly. Therefore, part of your job in this Capstone is to be a strategic thinker. You must decide when an agent adds enough value to be worth the extra cost, complexity, and time.

**Considerations for using standard prompts vs. agents**



## Safety and security in action

When we give AI models the power to take action in the real world, we must also think deeply about safety. In this course, we will discuss "human-in-the-loop" workflows. This means designing your agent so that it explicitly asks for human permission before taking a sensitive action. For example, the travel bot might find the hotel and fill out the booking form, but it should stop and ask the customer, "I have prepared the booking for Hotel Lumiere. Please confirm these details are correct before I submit the payment."

## Your capstone project overview

This course is structured to guide you step-by-step through building a complete project.

First, you will focus on the build. You will examine the syntax and mechanics of function calling. You will learn how to define tools and how to handle the data that comes back from them. You will design the architecture for your agent and create the conversation flow. By the end of this module, you will have a working application running locally on your machine.

Then, you'll shift your focus to production. Building an app that works on your laptop is one thing; putting it on the internet for the world to use is another challenge entirely. You will look at deployment planning. You will learn how to estimate costs and monitor your app to make sure it stays healthy. You will examine how to deploy your application to the cloud so it can run reliably 24/7.

## Key takeaways

- AI is evolving rapidly, creating a strong need for adaptability and continuous learning.
- Concepts like function calling and agent architecture are high-demand skills that employers are actively seeking.
- This project serves as an opportunity to build unique and practical applications.

Now that you understand the API basics and how to prompt the model, you have the necessary tools to apply your knowledge and begin building.

Mark as completed

Like Dislike Report an issue

Go to next item →